Reviewer Pack — Lorentzian Defect of Cut Polynomial Lower-Bounds Max-CUT SoS…

Entry dad52f056827 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:13:21 UTC

Lorentzian Defect of Cut Polynomial Lower-Bounds Max-CUT SoS-2 Gap

Entry ID: `dad52f056827` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 02:13:21 UTC

1. Conjecture

Field A (mathematical branch): Lorentzian polynomials / Brändén–Huh log-concavity and the matroid Hodge–Riemann relations of Adiprasito–Huh–Katz — the normalized log-concavity condition $\tilde{a}_{j-1}\cdot\tilde{a}_{j+1} \le \tilde{a}_j^2$ with $\tilde{a}_j := a_j / C(m,j)$ characterizing when a homogenized univariate coefficient vector lies in the closure of the Lorentzian cone; the defect $LD := \max_j \max(0, \log(\tilde{a}_{j-1}\cdot\tilde{a}_{j+1}/\tilde{a}_j^2))$ is a sup-of-log-ratios functional that uses only integer counts, binomial normalisation, and log of ratios, with no F_q polynomial extension (binomial normalisation and log are not preserved under polynomial ring lifts), so the bridge is Aaronson–Wigderson algebrization-safe. Distinct from blacklisted Lee–Yang zero cluster angle (complex roots of the same polynomial), Kashin L^1-flatness (single-eigenvector L^1/L^2 ratio), Curto–Fialkow Hankel-apolar rank (real moment problem on L_G spectrum), free 4-cumulant excess (Voiculescu NC partitions on squared singular values), Fekete capacity (transfinite diameter of Laplacian spectrum), hypercontractive 4/2 ratio (norm ratio of the truth table), and Schur–Horn majorization gap (NATURAL_PROOFS-rejected). An arXiv search 'Lorentzian polynomial' AND ('Max-Cut' OR 'Delorme–Poljak' OR 'SoS integrality gap') returns 0 direct hits with <5 adjacent papers (Anari–Liu–Oveis-Gharan–Vinzant on log-concavity for matroid-base sampling, never on SoS-2 integrality gaps). **Field B** (complexity object): Degree-2 Sum-of-Squares / Delorme–Poljak Max-CUT integrality gap $\rho(G) := n\cdot\lambda_{max}(L_G)/(4\cdot MC(G)) - 1$ for connected 3-regular graphs G, in the Goemans–Williamson 1995 / Delorme–Poljak 1993 / Schoenebeck 2008 / Barak–Steurer 2014 SOS_HIERARCHY regime; the canonical stepping stone toward unconditional SoS-degree-d Max-CUT lower bounds via matroidal obstructions on degree-2 pseudoexpectation moment matrices.

Statement:

> Let G be a connected 3-regular graph on n vertices with $m = 3n/2$ edges and cut-size distribution $a_j(G) := \#\{S \subseteq V(G) : |\delta_G(S)| = j\}$, and let $LD(G) := \max\{0, \max_{1\le j\le m-1, a_{j-1}a_ja_{j+1}>0} \log((\tilde{a}_{j-1}\cdot\tilde{a}_{j+1})/\tilde{a}_j^2)\}$ with $\tilde{a}_j := a_j / C(m,j)$ be the normalized Lorentzian defect of the cut-generating polynomial $P_G(z) = \Sigma_j a_j z^j$. Then for every such G, $LD(G) \ge 0.05 \cdot \rho(G)$, where $\rho(G) = n\cdot\lambda_{max}(L_G)/(4\cdot MC(G)) - 1$ is the Delorme–Poljak SoS-2 integrality gap. Equivalently, any connected 3-regular G whose cut-polynomial coefficient sequence is normalized log-concave (Lorentzian, LD=0) must have $\rho(G) = 0$ (SoS-2 tight).

Rationale (proposer's reasoning):

The Brändén–Huh Lorentzian property of a homogeneous polynomial is exactly the matroidal Hodge–Riemann signature condition; if P_G(z,w) (homogenized) were Lorentzian, the symmetric bilinear form on (R^V, J_G) induced by the cut-direction Gram would inherit a one-positive-eigenvalue signature compatible with the Laplacian's spectral gap, forcing the SoS-2 pseudoexpectation cone to align with the integer-cut cone. Conversely the Delorme–Poljak gap measures the angle between the leading Laplacian eigenvector and the cut polytope's max-cut face, an angle that combinatorially obstructs the Newton-inequality regularity of (a_j). Bridging matroid log-concavity to the SoS-2 obstruction is essentially unexplored: it bypasses the NATURAL_PROOFS trap that killed Schur–Horn (the invariant is structural on the graph, not on a truth table) and the ALGEBRIZATION trap that closes ring-only constructions (binomial normalisation and log of ratios admit no F_q polynomial surrogate).

Taxonomy category: `COMM_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e6ebab19010cecce`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 210 instances (7 sizes n∈{8,...,20} × 30 seeded random connected 3-regular graphs), compute ρ(G) and LD(G) as specified. SUPPORTED iff every instance satisfies LD(G) ≥ 0.05·ρ(G); FALSIFIED iff any instance has ρ(G) ≥ 0.05 and LD(G) < 0.05·ρ(G).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.84	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Lorentzian polynomial cut polynomial Max-Cut integrality gap` - `normalized log-concave cut distribution Delorme Poljak SoS` - `log-concavity cut polynomial 3-regular graph eigenvalue Max-Cut`

Top relevant hits considered: - [s2:2407.19905] The Bidirected Cut Relaxation for Steiner Tree has Integrality Gap Smaller Than 2 - [s2:10.4230/LIPIcs.SWAT.2024.23] A Logarithmic Integrality Gap for Generalizations of Quasi-Bipartite Instances of Directed Steiner Tree

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        edges = set()
        while len(edges) < 3 * n // 2:
            u, v = random.sample(range(n), 2)
            if (u, v) not in edges and (v, u) not in edges:
                edges.add((u, v))
        return list(edges)

    def popcount(x):
        count = 0
        while x:
            count += x & 1
            x >>= 1
        return count

    def laplacian_eigenvalues(n, m):
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            L[i][i] = 2
        for u, v in edges:
            L[u][v] = -1
            L[v][u] = -1
        return [eigenvalue for eigenvalue in numpy.linalg.eigvalsh(L) if eigenvalue >= 0]

    def compute_a_j(n, m):
        a_j = defaultdict(int)
        for S in range(2 ** n):
            mask = [S >> i & 1 for i in range(n)]
            degree_sum = sum(mask[u] ^ mask[v] for u, v in edges if (u, v) in edges or (v, u) in e
            a_j[degree_sum] += popcount(S)
        return {j: a / math.comb(m, j) for j, a in a_j.items()}

    def compute_rho(n, lambda_max, MC):
        return n * lambda_max / (4 * MC) - 1

    def compute_LD(a_j):
        m = len(edges)
        LD = 0
        for j in range(1, m):
            if a_j[j-1] > 0 and a_j[j] > 0 and a_j[j+1] > 0:
                LD = max(LD, math.log((a_j[j-1] * a_j[j+1]) / (a_j[j]**2)))
        return LD

    n_values = [8, 10, 12, 14, 16, 18, 20]
    results = []

    for n in n_values:
        m = 3 * n // 2
```

```
        edges = generate_graph(n)
        a_j = compute_a_j(n, m)
        MC = max(a_j.values())
        lambda_max = laplacian_eigenvalues(n, m)[0]
        rho = compute_rho(n, lambda_max, MC)
        LD = compute_LD(a_j)

        results.append({
            "n": n,
            "m": m,
            "rho": rho,
            "LD": LD
        })

    all_hold = True
    counterexample = ""
    for result in results:
        if result["LD"] < 0.05 * result["rho"]:
            all_hold = False
            counterexample = f"n={result['n']}, m={result['m']}, rho={result['rho']:.4f}, LD={resu

    return {
        "metric_name": "Lorentzian Defect",
        "metric_value": sum(result["LD"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all_hold,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std=0.0000 support_fraction={support_fract
    elif any(not result["conjecture_holds"] and result["LD"] < 0.05 * result["rho"] for result in
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=n={result['n']}, m={result['m']}, rho={result['rh
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b1bc8fd9.py", line 103, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b1bc8fd9.py", line 71, in run_trial
    lambda_max = laplacian_eigenvalues(n, m)[0]
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b1bc8fd9.py", line 42, in laplacian_eig
    return [eigenvalue for eigenvalue in numpy.linalg.eigvalsh(L) if eigenvalue >= 0]
                                         ^^^^^
NameError: name 'numpy' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a NameError ('numpy' not defined) before any of the 210 instances were evaluated, so no data on the LD(G) ≥ 0.05·ρ(G) inequality was produced. Per Rule 2, a crashed test yields INCONCLUSIVE. | next: Fix the import (add 'import numpy' or use the already-imported alias) in laplacian_eigenvalues and rerun the full 210-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	165083
2	preregistration	claude_max	opus	0	0	5382
3	novelty	claude_max	opus	0	0	3197
4	novelty	claude_max	opus	0	0	6205
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12639
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16638
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12476
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12956
9	judge	claude_max	opus	0	0	5789

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 240365 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/dad52f056827.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dad52f056827.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dad52f056827.tar.gz` (if generated)